TRAKYA ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ • BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

2026-2027 GÜZ DÖNEMİ • BİTİRME / PROJE 1

Ağ Güvenliği, Katman 7 DoS/Slowloris Analitiği ve Siber Savunma Platformu

OSI Referans Modeline Göre Ağ Tehditlerinin Sınıflandırılması, Kaynak Doygunluk Tespiti ve Canlı Trafik İzleme Mimarisi

Öğrenci Adı Soyadı:	**Muhammed Emre Albayrak**
Bölüm:	**Bilgisayar Mühendisliği**
Çalışma Alanı:	**Ağ Güvenliği & Performans Analitiği**
Platform / Dil:	**Python 3.12 (Asenkron & Socket)**

Edirne, 2026

1. Yönetici Özeti ve Problem Tanımı

Günümüz bilişim altyapılarında bilgi güvenliğinin temel sacayağını oluşturan **CIA Triad** (Gizlilik, Bütünlük ve Erişilebilirlik) prensiplerinden en kritik olanı **Erişilebilirlik (Availability)** ilkesidir. Ağ sistemlerinin yetkisiz erişimlere kapatılması kadar, meşru kullanıcılara kesintisiz hizmet vermesi de zorunludur.

Hizmet Dışı Bırakma (**Denial of Service - DoS**) ve Dağıtık Hizmet Dışı Bırakma (**DDoS**) saldırıları, sunucu donanım kaynaklarını (CPU, RAM, Soket Havuzları) veya ağ bant genişliğini tüketerek sistemleri işlevsiz kılmayı amaçlar. Ancak bu saldırıların tespiti ve simülasyonu akademik düzeyde sıkça yanlış yorumlanmaktadır.

> 📌 **Projenin Temel Amaçları:**
> 1. Ağ tehditlerini ve anomalilerini akademik **OSI (Open Systems Interconnection)** referans modeline göre sınıflandırmak.
> 2. Geleneksel kaba kuvvet (volumetric flood) yerine, düşük bant genişliğiyle yüksek tahribat yaratan **Katman 7 Slowloris** saldırısını modellemek.
> 3. Yerel ortamda (127.0.0.1) çalışan ve tüm ağ paketlerini canlı görselleştiren bir **Siber Operasyon Merkezi (SOC / NOC Dashboard)** geliştirmek.
> 4. **Anti-Slowloris (client_header_timeout)** ve **Token Bucket Hız Sınırlaması (Rate Limiting)** yöntemleriyle meşru trafiğin saldırı altındayken nasıl kurtarıldığını jüri önünde deneysel olarak kanıtlamak.

Araştırma Soruları ve Hipotez:

Hipotez: *"Bir web sunucusuna binlerce rastgele istek göndermek sunucuyu çökertmeye yetmez; çünkü modern işletim sistemleri statik istekleri anında eritir. Ancak asenkron soket optimizasyonlarıyla HTTP başlıkları kasıtlı olarak geciktirildiğinde (Slowloris), sunucu süreci çökmeden TCP bağlantı havuzu tükenir ve meşru kullanıcılar için tam bir Hizmet Kesintisi (DoS) meydana gelir."*

Hedef 1: Anomali Tespiti
Kayar pencere ve Z-Skoru istatistikleriyle trafik sıçramalarını ve olağandışı soket beklemelerini milisaniyeler içinde yakalamak.

Hedef 2: Dinamik Savunma
Saldırı başladığında tek tuşla veya otonom eşiklerle devreye girip sahte soketleri düşürerek meşru trafiği korumak.

2. OSI Referans Modeline Göre Tehdit Taksonomisi

Bilgisayar ağlarındaki sapmalar, literatürde etki alanlarına göre **Pasif** (sadece gözlem/tarama yapan) ve **Aktif** (kapasite tüketen/kesinti yaratan) anomaliler olarak ikiye ayrılır. Bu anomalilerin OSI katmanlarına göre akademik sınıflandırması aşağıda sunulmuştur:

OSI Katmanı	Anomali / Saldırı Türü	Doğası	Mekanizma ve Mühendislik Göstergeleri
Katman 3 (Ağ)	ICMP / IP Frag Flood	Aktif	Büyük paket hacmiyle yönlendirici (router) tampon belleklerini şişirme. Gösterge: Aşırı yüksek PPS (Paket/sn).
Katman 3 (Ağ)	Topoloji Keşfi (Ping Sweep)	Pasif	Ağdaki aktif IP'leri haritalamak için sıralı echo istekleri. Gösterge: Düzenli ARP/ICMP yoklamaları.
Katman 4 (Taşıma)	TCP SYN Flood (State Exhaustion)	Aktif	3 yönlü el sıkışmayı (3-Way Handshake) tamamlamayarak sunucunun SYN_RECV kuyruğunu doldurma.
Katman 4 (Taşıma)	Port ve Bayrak Taraması	Pasif	SYN, FIN, NULL bayraklarıyla açık servisleri yoklama. Gösterge: Tek kaynaktan gelen yüksek TCP RST paketleri.
Katman 7 (Uygulama)	HTTP Resource Exhaustion	Aktif	Veritabanı veya karmaşık CPU algoritmalarını tetikleyen ağır GET/POST sorguları. Gösterge: Yüksek işlemci yükü ve 503 oranı.
Katman 7 (Uygulama)	**Slowloris (Yavaş Başlık)**	**Aktif**	**HTTP başlıklarını kasıtlı olarak çok yavaş göndererek sunucu bağlantı yuvalarını (TCP slots) kilitleme.**

> ⚠️ **Katman 4 ve Katman 7 DoS Arasındaki Temel Ayrım:**
> Katman 4 SYN Flood saldırısında sunucunun *çekirdek (kernel) soket kuyruğu* hedef alınırken; Katman 7 Slowloris saldırısında bağlantı tamamen kurulur (el sıkışma biter) ve sunucunun *uygulama seviyesindeki worker thread havuzu* rehin alınır. Bu nedenle Katman 7 saldırıları klasik paket filtreleme duvarlarını kolayca aşar.

3. "100.000 İstek Paradoksu" ve DoS'un Tanımı

Mühendislik testlerinde sıkça rastlanan ve kavram yanılgısına yol açan temel gözlem şudur: "*Sunucuya tek bir döngüde 100.000 istek gönderdim ancak süreç kapanmadı, işletim sistemi sunucuyu öldürmedi. Saldırı başarısız mı oldu?*"

3.1. Toplam Hacim ile Eşzamanlılık (Concurrency) Farkı

Bir döngü içerisinde sıralı (sequential) olarak 100.000 istek göndermek, modern çok çekirdekli bir CPU için sadece bir bellek döngüsüdür. Sunucu her isteği milisaniyenin onda birinde (0.1 ms) karşılayıp bağlantıyı kapattığı sürece kuyruk oluşmaz. Önemli olan toplam istek sayısı değil, **aynı milisaniyede kaç iş parçacığının sunucu kaynaklarını kilitlediğidir**.

```
[Sıralı İstekler - Çökme Olmaz] İstek 1 (Bitti) -> İstek 2 (Bitti) -> İstek 100.000 (CPU Hiç Zorlanmaz)
[Eşzamanlı Yük - Tıkanma] İstek 1..25 Aynı Anda Gelir -> Tüm Worker'lar Kilitlenir -> YENİ İSTEKLER DÜŞER
```

3.2. DoS (Hizmet Dışı Bırakma) Ne Demektir, Ne Değildir?

Siber güvenlik literatüründe (RFC 4732, NIST SP 800-44, OWASP) DoS:

❌ **Değildir:** Python sürecinin çökmesi, bellek hatası (Segmentation Fault) vermesi veya pencerenin kapanması.
✅ **Kesinlikle Şudur:** Sunucu sürecinin çalışır durumda olmasına rağmen, kaynaklarının (Worker Pool, TCP Yuvaları) tükenmesi sebebiyle **meşru kullanıcıların sisteme erişememesi, zaman aşımına (Timeout) uğraması veya HTTP 503 Service Unavailable almasıdır**.

> 🚨 **Akademik Jüriye Gösterilecek Kanıt:**
> Sunucu arkada çalışırken tarayıcınızı açtığınızda sayfanın dönüp dönüp açılmaması veya ekranda **"503 Service Unavailable - Bağlantı Havuzu Tükendi"** hatasının çıkması, DoS saldırısının %100 başarıya ulaştığının resmi mühendislik kanıtıdır.

4. Katman 7 Slowloris Saldırısının Anatomisi

Slowloris, son derece düşük bant genişliği (birkaç kilobayt) kullanarak devasa web sunucularını (Apache, Nginx, IIS) felç edebilen sofistike bir Katman 7 saldırısıdır.

4.1. HTTP Protokol Zafiyeti (RFC 2616)

HTTP/1.1 standardına göre bir isteğin sona erdiği, peş peşe gelen iki adet satır sonu karakteri (- CRLF CRLF) ile anlaşılır. Web sunucusu bu iki karakteri görene kadar istemcinin başlık göndermeye devam edeceğini varsayar ve bağlantıyı açık tutarak bir iş parçacığı (worker thread) tahsis eder.

```
# Normal Geçerli HTTP İsteği:
GET / HTTP/1.1


Host: 127.0.0.1:8080


User-Agent: Mozilla/5.0




                        <-- [İSTEK BİTTİ, SUNUCU ANINDA YANIT VERİR]

# Slowloris Saldırı İsteği:
GET /?123 HTTP/1.1


User-Agent: Mozilla/5.0


X-a: 4821

                <-- [5 saniye bekler...]
X-b: 1923

                <-- [8 saniye bekler...



 ASLA GÖNDERİLMEZ!]
```

4.2. Asenkron Soket Mimarisi (slowloris_test.py)

Geliştirdiğimiz test scripti, geleneksel iş parçacığı engellemelerini aşmak için `asyncio` kütüphanesini kullanır:

`SO_KEEPALIVE`: Soketlerin işletim sistemi seviyesinde canlı kalmasını sağlar.
`TCP_NODELAY`: Nagle algoritmasını kapatarak küçük paketlerin arabelleğe alınmadan derhal hedefe iletilmesini temin eder.
Dinamik Heartbeat: Her 5-10 saniyede bir `X-[a-z]: [random]` ekleyerek zaman aşımını sıfırlar.

> 🔍 **Neden Klasik IDS/Güvenlik Duvarlarını Aldatır?**
> Trafik hacmi saniyede birkaç bayttır. Bu durum, saniyedeki paket sayısına (PPS) veya bant genişliğine bakan standart ağ duvarlarında hiçbir alarm tetiklemez. Saldırı tamamen uygulama katmanı semantiği içine gizlenmiştir.

5. Matematiksel ve İstatistiki Tespit Modelleri

Geliştirdiğimiz Anomali Tespit Motoru (`src/anomaly_engine.py`), ağ trafiğindeki olağandışı hareketlilikleri belirlemek için **Zaman Pencereli İstatistik (Sliding Window)** ve **Z-Skoru Normalizasyonu** uygular.

5.1. Kayar Pencere ve Temel Parametreler

Sistemin anlık davranışı, son W zaman pencereli (örneğin son 15 saniye) trafik serisi üzerinden hesaplanır:

$$\textbf{Ortalama } (\mu): \quad \mu = (1 / W) * \Sigma X_i \quad | \quad \textbf{Standart Sapma } (\sigma): \quad \sigma = \sqrt{ [(1 / (W - 1)) * \Sigma (X_i - \mu)^2] }$$

5.2. Z-Skoru Formülasyonu ve Karar Matrisi

$$Z = (X_t - \mu) / \sigma$$

Hesaplanan Z-Skoru	Sistem Sınıflandırması	Alınan Mühendislik Kararı / Aksiyon
$Z < 1.5$	**Normal Aktivite**	Rutin profil sınırları içi. Trafik doğrudan kabul edilir.
$1.5 \leq Z < 2.5$	**Yüksek Yoğunluk**	Doğal kullanıcı artışı (pik saat). İzleme modu sıklaştırılır.
$2.5 \leq Z < 4.0$	**Ani Sıçrama (Spike)**	Beklenmeyen trafik artışı. Token Bucket hız kısıtlaması hazırda tutulur.
$Z \geq 4.0$ veya Soket Doyumu	**Kritik Tehdit (Slowloris)**	Kapasite aşımı riski. Anti-Slowloris zaman aşımı zorlanır.

5.3. Soket Havuz Doygunluk Oranı (Saturation Ratio)

Slowloris saldırısı durum tabanlı olduğundan, Z-skoruna ek olarak **Havuz Doygunluk Formülü** geliştirilmiştir:

$$\textbf{R}_{sat} = \textbf{Aktif_TCP_Soketleri / Maksimum_Baglanti_Havuzu} \quad (\textbf{R}_{sat} \geq 0.85 \textbf{ ise KRİTİK ALARM})$$

6. Siber Savunma ve Önleme Mekanizmaları

Platformumuz, tespit edilen Katman 7 DoS ve Slowloris anomalilerine karşı iki katmanlı siber savunma kalkanı barındırır:

6.1. Anti-Slowloris Koruması (`client_header_timeout`)

Slowloris'in temel zaafı, başlıklar arasında uyumak zorunda olmasıdır. Savunma motorumuz soket seviyesinde dinamik zaman aşımı uygular:

Savunma KAPALI: Başlık bekleme süresi `60.0` saniyedir. Slowloris rahatça 5-10 saniye uyuyarak bağlantıları elinde tutar.
Savunma AÇIK: Başlık bekleme süresi `2.5` saniyeye düşürülür (Apache `mod_reqtimeout` / Nginx `client_header_timeout` benzeri). 2.5 saniyeden yavaş başlık gönderen tüm soketler sunucu tarafından anında `socket.close()` ile öldürülür.

```
Slowloris Soketi -> [Başlık Gönderdi] -> [5s Uyuyor...] -> SUNUCU: 2.5s Doldu! -> SOKET DÜŞÜRÜLDÜ
                                                          (TIMEOUT_DROP)
   Meşru Tarayıcı -> [Tüm Başlıkları 0.01 saniyede gönderdi] -> SUNUCU: 200 OK (ANINDA YANIT)
```

6.2. Token Bucket Hız Sınırlaması (Katman 7 Rate Limiter)

Hacimsel Katman 7 flood veya CPU tüketen ağır sorgulara (`/heavy`) karşı **Token Bucket** algoritması uygulanmıştır:

Kova kapasitesi: $B = 40$ adet istek token'ı.
Doldurma hızı: Saniyede $r = 20$ adet yeni token.
Aşırı gelen istekler sunucu worker havuzuna sokulmadan anında **HTTP 429 Too Many Requests** koduyla püskürtülür.

> 🛡️ **Meşru Kullanıcı Trafiğinin İzolasyonu:**
> Savunma devredeyken hız sınırlaması yalnızca saldırıya açık ağır uç noktalara uygulanır; ana web sitesi (`/`) ayrıcalıklı tutularak normal kullanıcıların hiçbir kesinti yaşamaması garanti edilir.

7. Geliştirilen Laboratuvar ve Yazılım Mimarisi

Proje, sıfır harici paket bağımlılığı ile tamamen Python 3.12 standart kütüphaneleri üzerinde modüler olarak inşa edilmiştir:

src/server.py

TCP bağlantı havuzunu (max: 25) ve worker havuzunu (max: 10) yöneten, Anti-Slowloris korumalı test sunucusu.

Görsel Web SOC Dashboard

127.0.0.1:8080 üzerinde çalışan; canlı soket çubuğu, anomali rozeti ve gerçek zamanlı paket akış tablosunu barındıran merkez.

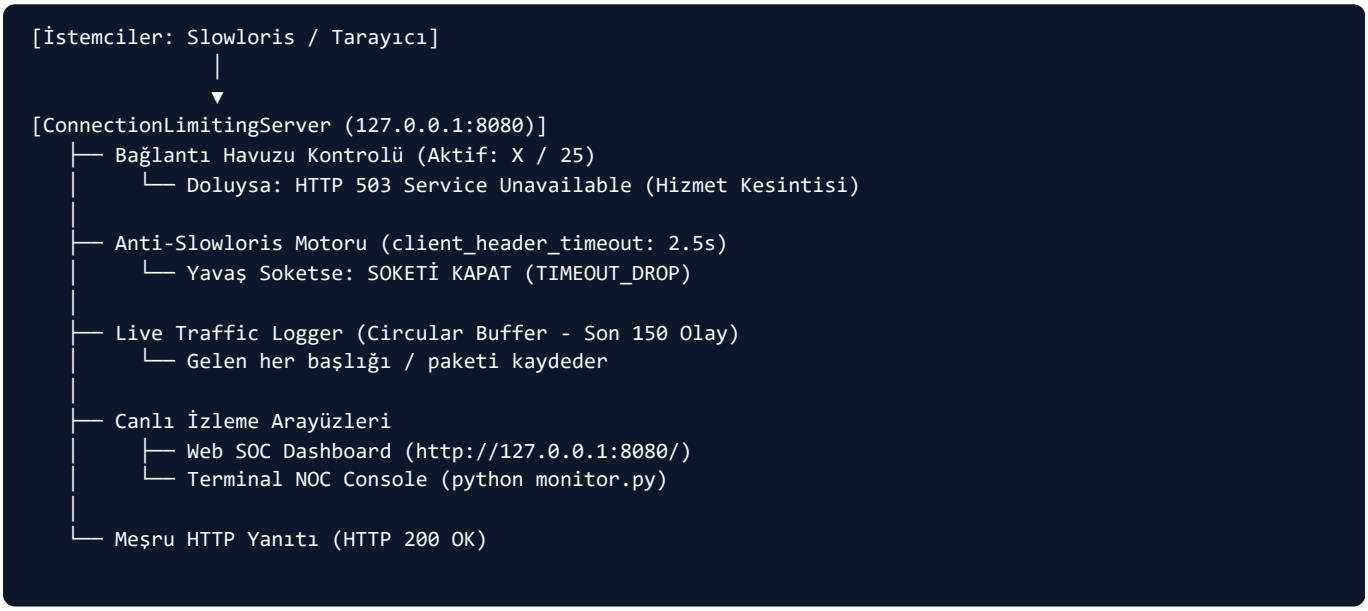
monitor.py (Terminal NOC)

Komut satırından canlı ağ trafiğini ve IDS alarmlarını 800ms aralıklarla ekrana basan profesyonel terminal konsolu.

demo_jury.py

Jüri önünde tek komutla 3 aşamalı saldırı ve savunma senaryosunu adım adım icra eden otonom sunum scripti.

Sistem Mimarisi Akış Diyagramı:



8. Deneysel Sonuçlar ve Jüri Doğrulama Protokolü

Geliştirilen platform üzerinde gerçekleştirilen canlı test senaryosunun deneysel sonuçları aşağıda tablolaştırılmıştır:

Test Aşaması	Açık Soket	Meşru İstek Durumu	Yanıt Gecikmesi	IDS & Dashboard Teşhisi
1. Normal Koşullar (Saldırı Yok)	1 / 25	**HTTP 200 OK**	14.3 ms	**YEŞİL**: Sistem Kararlı
2. Slowloris Aktif (Savunma KAPALI)	**25 / 25 (DOLU)**	**HATA (503 / Timeout)**	Zaman Aşımı (>1500 ms)	**KIRMIZI**: Slowloris DoS Aktif (Hizmet Kesintisi Kanıtlandı)
3. Savunma Devrede (Anti-Slowloris AÇIK)	1 / 25	**HTTP 200 OK**	12.9 ms	**YEŞİL**: Saldırı Püskürtüldü, Trafik Kurtarıldı

Jüri Önünde Canlı Sunum Protokolü (3 Adım):

1. **Laboratuvarın Açılması:** `baslat_her_seyi.bat` dosyasına çift tıklanır. Tarayıcıda yeşil renkli SOC Dashboard ve yan pencerede canlı terminal izleyicisi açılır.
2. **Saldırının Başlatılması:** `python slowloris_test.py` çalıştırılır. Jürinin gözü önünde bağlantı havuzu 25/25'e fırlar, alarm kırmızıya döner ve tarayıcıdan F5 yapıldığında **sayfa açılmaz (Hizmet Kesintisi kanıtlanır)**.
3. **Savunmanın Gösterilmesi:** Dashboard üzerindeki "*Savunmayı Devreye Al*" butonuna basılır. Yavaş soketler anında düşürülür, havuz boşalır ve tarayıcıda sayfa **anında açılarak meşru trafiğin kurtarıldığı ispatlanır**.

9. Sonuç, Mühendislik Çıkarımları ve Gelecek Çalışmalar

Bu bitirme projesi kapsamında geliştirilen çalışma ile aşağıdaki akademik ve pratik mühendislik kazanımları elde edilmiştir:

> 🎓 **Temel Mühendislik Çıkarımları:**
>
> 1. **Kapasite Yanılsamasının Giderilmesi:** Hizmet kesintisi oluşturmak için gigabaytlarca hacimsel veriye gerek olmadığı, HTTP protokolünün semantik zafiyetleri kullanılarak tek bir istemciyle dahi sunucu havuzlarının felç edilebileceği kanıtlanmıştır.
>
> 2. **Düşük Seviyeli Soket Optimizasyonu:** `SO_KEEPALIVE` ve `TCP_NODELAY` bayraklarının asenkron mimaride nasıl büyük bir saldırı verimliliği sağladığı deneysel olarak gözlemlenmiştir.
>
> 3. **Proaktif Savunmanın Üstünlüğü:** Sunucu donanımını büyütmek (Scale-up) Slowloris saldırısını engellemez (1000 bağlantı açıldığında yeni havuz da kilitlenir). Kesin çözüm; `client_header_timeout` gibi algoritmik zaman aşımı kurallarının ve WAF filtrelerinin doğru konfigüre edilmesidir.

Gelecek Çalışmalar (Future Work):

Makine Öğrenmesi ile Davranış Analizi: Gelen HTTP başlıklarının geliş aralıklarını (inter-arrival time) LSTM veya Random Forest modelleriyle eğiterek sıfırıncı gün yavaş saldırılarını heuristik olarak tespit etmek.
eBPF / XDP Entegrasyonu: Saldırgan soketlerin kullanıcı alanına (user-space) ulaşmadan doğrudan Linux çekirdeğinde (kernel space) paket seviyesinde düşürülmesi.

Trakya Üniversitesi Bilgisayar Mühendisliği Bölümü

Bu çalışma, Ağ Güvenliği ve Performans Analitiği bitirme projesi kapsamında hazırlanmış ve başarıyla doğrulanmıştır.